Guía autoguiada · Practicar SQL en parejas (scripts 01–06)

PARA QUÉ SIRVE

os explica **cómo trabajar vosotros dos, paso a paso**, los seis scripts de SQL del Sprint 3, sin que nadie tenga que deciros qué hacer en cada momento.

- **Tiempo en clase:** unos **100 minutos** (el trabajo recomendado está en la tabla de la sección 4). Los ejercicios que no os dé tiempo a hacer se pueden terminar en casa.
- **Qué necesitáis: DB Browser for SQLite** instalado, el esquema de turisdata.db, el archivo `turisdata.db` y los scripts `sql/`. Un ordenador por pareja es suficiente.
- **Qué no necesitáis:** saber programar. SQL se parece a **hacer preguntas en inglés muy sencillo** a una tabla.
- **Datos:** son **sintéticos** (inventados). Sirven para practicar, no son estadísticas reales del turismo.

1. Antes de empezar (5 minutos)

1. **Formad parejas** y decidid quién **conduce** primero (el otro **navega**; sección 2).
2. **Conseguid y abrid la base de datos.** Descargad `turisdata.db` desde GitHub con el botón **Download raw file** (icono de flecha hacia abajo, arriba a la derecha de la página del archivo), o descargad todo el repositorio con **Code → Download ZIP**. En DB Browser: *Abrir base de datos* (Ctrl+O) → `turisdata.db`.
   - **Comprobad el tamaño del archivo:** debe pesar **unos 800 KB (815.104 bytes)**. Si pesa 0 KB o solo unos pocos KB, **la descarga ha fallado** (por ejemplo, si habéis guardado la página web de GitHub en lugar del archivo). Volved a descargarlo.
   - **Si se abre pero sale vacía:** mirad la pestaña *Estructura de la base de datos*. Deben aparecer **tres tablas** (`alojamientos`, `clientes`, `reservas`). Si no hay ninguna, usad la copia en texto: `turisdata.sql` (explicación en su primera línea): *Archivo → Nueva base de datos* (guardadla como `turisdata.db`, y **cancelad** el asistente de crear tabla) → *Archivo → Importar → Base de datos desde archivo SQL…* → `turisdata.sql`.
3. **Mirad las tres tablas.** Pestaña *Estructura de la base de datos*: veréis `alojamientos`, `clientes` y `reservas`. Desplegad cada una y leed los nombres de las columnas. **Tened abierto el esquema de turisdata.db**: dice qué campos tiene cada tabla y qué significan, para saber dónde buscar cada dato.
4. **Abrid el primer script en la pestaña *Ejecutar SQL*.** Los scripts están en la carpeta `sql/` (`01_select_where.sql` … `06_modelado_indices.sql`). Descargadlos igual que la base de datos (*Download raw file*) o con el ZIP del repositorio. Luego, en DB Browser: pestaña *Ejecutar SQL* → icono de la **carpeta** (*Abrir archivo SQL*) → elegid el script.
   - **Dónde está el listado:** en `listado_ejercicios_sql.md` tenéis los 50 ejercicios en tablas, por script, con casilla para ir marcando.
   - **Dónde están los enunciados:** dentro del propio script. Cada ejercicio es un comentario que empieza por `-- EJERCICIO 1.1:` (con su `-- PISTA:` debajo) y una línea `-- (escribe aquí tu consulta)`. **Escribid vuestra consulta justo debajo** de su enunciado. En total son **50 ejercicios**: 10 en el script 01, 7 en el 02, 8 en el 03, 8 en el 04, 9 en el 05 y 8 en el 06.
   - Los ejemplos de arriba de cada script (las «IDEAS») ya están escritos: probadlos antes de empezar los ejercicios.
   - Para ejecutar **un solo ejercicio**, seleccionad con el ratón solo su consulta y pulsad **Ctrl+Intro**. Si no seleccionáis nada, se ejecuta el script **entero**.
5. **Haced la prueba de arranque.** Escribid esta consulta, **seleccionadla con el ratón** y pulsad **Ctrl+Intro**:

```
SELECT COUNT(*) FROM reservas;
```

Debería salir **6000**. Si sale otro número o un error, parad y preguntad antes de seguir: algo no está bien abierto.

Cifras de control (para saber que estáis en la base correcta):

Qué contáis	Debe salir
Reservas	**6.000**
Reservas canceladas (`cancelada = 1`)	**2.754** (el 45,9 %)
Alojamientos	**20**
Clientes	**2.500**

2. Cómo trabajáis en pareja: conductor y navegante

Rol	Qué hace	Qué no hace
Conductor/a	Tiene el teclado. **Escribe** la consulta que decidís entre los dos.	No decide solo: explica en voz alta lo que escribe.
Navegante	**Lee** el enunciado y la pista, **piensa en voz alta**, **revisa** lo que se escribe y comprueba el resultado.	No toca el teclado ni dicta letra por letra. Propone ideas y pregunta.

Reglas de oro

- **Cambiad de rol en cada ejercicio** (o cada 10 minutos). Quien conduce primero navega después.
- **Nadie se queda mirando.** Si una persona no entiende algo, **se para** hasta que lo entienda: explicarlo también es aprender.
- **Hablad en voz alta.** «Voy a pedir el nombre y la isla de la tabla alojamientos, solo los de Tenerife» es mejor que escribir en silencio.
- **Equivocarse es normal.** Un mensaje de error es **una pista**, no un fallo vuestro.
- **Pedid ayuda así:** primero preguntáos entre vosotros, luego a la pareja de al lado, y **después** a vuestro docente. Llevad **la consulta que habéis probado y el mensaje de error**.

3. El ciclo de cada ejercicio (6 pasos)

Repetid esto **en cada ejercicio**. Con la práctica os llevará menos de 3 minutos.

1. **Leed** el enunciado **en voz alta**, los dos.
2. **Predecid** antes de escribir: *¿qué columnas quiero ver? ¿de qué tabla? ¿cuántas filas más o menos espero (pocas, muchas, una)?*
3. **Escribid poco a poco**, de la tabla hacia fuera: primero `FROM` (de dónde), luego `SELECT` (qué columnas), luego `WHERE` (qué filas), y al final el orden o el recorte. **Ejecutad pronto y a menudo**, aunque la consulta esté a medias.
4. **Ejecutad solo esa consulta:** seleccionadla con el ratón y **Ctrl+Intro**. (Si no seleccionáis nada, DB Browser ejecuta **todo el script de golpe**.)
5. **Comprobad** el resultado con la lista de abajo.
6. **Explicad a vuestra pareja** con una frase qué hace cada línea. Si no sabéis explicar una línea, **todavía no la habéis entendido**.

Lista de comprobación (paso 5)

- [ ] **¿Aparecen las columnas que pedía el enunciado**, y solo esas?
- [ ] **¿Cuántas filas salen?** DB Browser lo dice abajo. ¿Coincide con lo que predijisteis?
- [ ] **¿Los valores tienen sentido?** (precios positivos, islas que existen, fechas del año correcto…)
- [ ] **¿Hay `NULL` o repetidos** donde no debería haberlos?

- [ ] Si usáis filtros de texto: **¿está escrito exactamente igual** que en la tabla, con comillas **simples**?
- [ ] Si habéis unido tablas (`JOIN`): **¿el número de filas es razonable?** Si sale una cifra enorme, el `ON` está mal.

Cómo saber si el resultado es correcto, **antes de mirar las soluciones**

- **Probad una versión más fácil.** Antes de pedir «canceladas del canal directo de más de 5 noches», probad solo «canceladas».
- **Contad.** `SELECT COUNT(*) ...` con el mismo `FROM` y `WHERE` os dice cuántas filas debería haber.
- **Comprobad con otro camino.** Si resolvéis algo de dos formas distintas (por ejemplo, el ejercicio 2.5 y el 4.4 piden lo mismo), **tienen que dar lo mismo**.
- **Sumad los grupos.** Si agrupáis con `GROUP BY` y sumáis los `COUNT`, **tiene que salir el total** (6.000 reservas en total).
- **Mirad algunas filas a ojo** con `LIMIT 5` para ver si coinciden con lo que pedía el enunciado.
- **Solo después**, abrid el solucionario paso a paso y comparad el número de filas y el valor que debe salir.

---

4. Plan de la sesión (100 minutos)

Script	Minutos	Qué hacéis (núcleo)	Qué es opcional
01 · `SELECT`, `WHERE`	15	Ejercicios **1.1 a 1.9**	**1.10** (reto)
02 · `JOIN`	15	**2.1 a 2.6**	**2.7** (reto)
03 · `GROUP BY`	20	**3.1 a 3.7**	**3.8** (reto)
04 · Subconsultas	15	**4.1 a 4.7**	**4.8** (reto)
05 · CRUD	15	**5.1 a 5.8**	**5.9** (reto)
06 · Modelado e índices	20	**6.1 a 6.7**	**6.8** (reto de diseño, en papel)

Si vais con prisa: haced **primero 01, 02 y 03 sin los retos**. Es lo mínimo imprescindible. Después, lo que podáis de 04, 05 y 06. **Si vais sobrados:** haced los retos y, para cada uno, **explicad a otra pareja** cómo lo habéis resuelto.

Control del tiempo. Poned un reloj a la vista. Si estáis **más de 5 minutos atascados** en un ejercicio, **pasad al siguiente** y volved después. Anotadlo en la hoja de la sección 8.

---

5. Cada script en una página

> Aquí tenéis **la idea de cada script**, un **esqueleto** y **qué se suele hacer mal**. **No** están las soluciones: las construís vosotros. Cuando hayáis terminado (o llevéis mucho rato atascados), podéis contrastar con el solucionario paso a paso.

Script 01 · Elegir columnas y filtrar filas

Idea clave: `SELECT` elige **columnas**; `WHERE` elige **filas**; `ORDER BY` **ordena**; `LIMIT` **recorta**.

Esqueleto:

```
SELECT columna1, columna2
FROM tabla
WHERE condición
ORDER BY columna DESC
LIMIT 5;
```

Se suele hacer mal:

- **Texto sin comillas simples** (`isla = Tenerife` → `isla = 'Tenerife'`) o con **mayúsculas distintas**.
- **Escribir `= NULL`.** Para datos ausentes se usa **`IS NULL`**.
- **Fechas:** son **texto** con formato `AAAA-MM-DD`, y se comparan como texto.

Comentad en pareja: *¿qué diferencia hay entre `WHERE` y `ORDER BY`?*

Script 02 · Unir tablas con JOIN

Idea clave: cada dato vive **en una sola tabla**, y `JOIN` las **une por una clave** (el `id` que comparten).

Esqueleto:

```
SELECT r.id_reserva, a.nombre, a.categoria
FROM reservas r
JOIN alojamientos a ON r.id_alojamiento = a.id_alojamiento
WHERE a.isla = 'La Palma';
```

(`r` y `a` son **alias**: nombres cortos para las tablas.)

Se suele hacer mal:

- **`ON` equivocado:** si une columnas que no son la clave, salen **miles de filas de más**.
- **Olvidar el alias** (`r.` o `a.`): sale `ambiguous column name` o `no such column`.
- **Confundir `JOIN` y `LEFT JOIN`:** `LEFT JOIN` **conserva todas las filas de la izquierda**, aunque no tengan pareja. Para encontrar «los que **no** tienen», se une con `LEFT JOIN` y se filtra con **`IS NULL`**.

Comentad en pareja: *¿cuándo usaríamos `LEFT JOIN` en lugar de `JOIN`?*

Script 03 · Resumir con GROUP BY

Idea clave: agrupar filas y **resumirlas** con `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. Como `cancelada` vale 0 o 1, **`AVG(cancelada)` es la proporción de canceladas**.

Esqueleto:

```
SELECT regimen, COUNT(*) AS total
FROM reservas
GROUP BY regimen
HAVING COUNT(*) > 1000
ORDER BY total DESC;
```

Se suele hacer mal:

- **`WHERE` frente a `HAVING`:** `WHERE` filtra **filas antes** de agrupar; `HAVING` filtra **grupos después**.
- **Poner en el `SELECT` una columna que no está en el `GROUP BY`** (ni dentro de una función como `COUNT`).
- **Los porcentajes:** `AVG(cancelada)` sale entre 0 y 1; **multiplicad por 100** y redondead con `ROUND(..., 1)`.

Comentad en pareja: *¿qué pasa si quito el `GROUP BY` de una consulta con `COUNT(*)`?*

Script 04 · Una consulta dentro de otra

Idea clave: a veces para responder una pregunta **necesitamos antes otra respuesta** (por ejemplo, «la media»). La subconsulta **calcula ese paso** y la consulta principal lo usa.

Esqueletos:

```
-- Comparar con un valor calculado
SELECT id_reserva, adultos
FROM reservas
WHERE adultos > (SELECT AVG(adultos) FROM reservas);

-- Comparar con una lista
SELECT nombre
FROM alojamientos
WHERE id_alojamiento IN (SELECT id_alojamiento FROM reservas WHERE precio_noche > 200);

-- Dar nombre a un paso intermedio (CTE)
WITH por_regimen AS (
    SELECT regimen, AVG(precio_noche) AS precio_medio FROM reservas GROUP BY regimen
)
SELECT * FROM por_regimen;
```

Se suele hacer mal:

- **Escribir todo de una vez. Probad primero la subconsulta sola** (seleccionadla y ejecutadla) y comprobad que da lo que esperáis.
- **Olvidar los paréntesis** de la subconsulta.
- **Una subconsulta que devuelve varias filas** donde solo cabe un valor.

Comentad en pareja: *¿qué hace la CTE que no hiciera una subconsulta?* (Pista: el **nombre**.)

Script 05 · Crear, modificar y borrar (CRUD)

Idea clave: `INSERT` **añade** filas, `UPDATE` **modifica**, `DELETE` **borra**. Cambiar datos es **peligroso**, así que trabajáis **sobre una copia** (`reservas_practica`) y nunca sobre las tablas originales.

Regla de oro (la más importante de toda la sesión):

> **Antes de un `UPDATE` o un `DELETE`, escribid el mismo `WHERE` en un `SELECT` y comprobad qué filas vais a tocar.** Un `UPDATE` o `DELETE` **sin `WHERE`** cambia o borra **toda la tabla**.

Esqueleto en tres pasos:

```
-- 1) Mirar qué filas toco
SELECT COUNT(*) FROM reservas_practica WHERE canal = 'agencia';
-- 2) Si el número tiene sentido, el mismo WHERE en el DELETE
DELETE FROM reservas_practica WHERE canal = 'agencia';
-- 3) Comprobar el resultado
SELECT COUNT(*) FROM reservas_practica;
```

Se suele hacer mal:

- **Ejecutar todo el script de golpe:** la preparación (`DROP TABLE` / `CREATE TABLE`) **vuelve a crear la copia** y borra vuestros cambios. **Seleccionad y ejecutad una consulta cada vez.**
- **Olvidar que los cambios no quedan guardados** hasta pulsar **Escribir cambios (Ctrl+S)**. Si algo sale mal: **Revertir cambios**.
- **Una transacción abierta** (`BEGIN`) sin `ROLLBACK` o `COMMIT`.

Comentad en pareja: *¿qué diferencia hay entre `ROLLBACK` y «Revertir cambios»?* (Los dos deshacen; `ROLLBACK` es una orden SQL.)

Script 06 · Diseñar tablas con reglas e índices

Idea clave: diseñar bien evita errores antes de que ocurran: la **clave primaria** identifica cada fila; la **clave foránea** obliga a que lo referenciado exista; `CHECK`, `NOT NULL` y `UNIQUE` son **reglas de calidad**; un **índice** acelera las búsquedas (como el índice de un libro).

Aquí hay errores que tienen que salir. Varios ejercicios (6.2, 6.3, 6.5) **piden que la inserción falle**, y **ese fallo es la respuesta correcta**. Ejecutad **esa sentencia sola**, leed el error y explicad **qué regla lo ha provocado**.

Esqueleto de una tabla con reglas:

```
CREATE TABLE p_ejemplo (
    id        INTEGER PRIMARY KEY,
    id_otra   INTEGER NOT NULL REFERENCES p_otra(id),
    puntos    INTEGER NOT NULL CHECK (puntos BETWEEN 1 AND 5)
);
```

Se suele hacer mal:

- **Ejecutar el script entero:** se **detiene** en la primera sentencia que debe fallar. Seguid **desde la siguiente**.
- **Crear una tabla que ya existe:** primero se borra con `DROP TABLE IF EXISTS`.
- **Dejar la limpieza sin hacer:** al terminar, **ejecutad la sección «LIMPIEZA»** del final del script para borrar las tablas de práctica (`p_...`).

Comentad en pareja: *¿por qué es bueno que la base de datos rechace una reserva de 0 noches?*

---

6. Si algo falla

Qué veis	Qué suele pasar	Qué hacer
`near "...": syntax error`	Falta una **coma** o un **punto y coma**, o hay una palabra mal escrita	Leed **la palabra que cita el error** y la línea de **justo antes**
`no such column: ...`	Nombre de columna mal escrito o **falta el alias** (`r.`, `a.`)	Haced `SELECT * FROM tabla LIMIT 5;` y comprobad los nombres
`ambiguous column name`	La columna existe en **dos tablas** y no habéis dicho de cuál	Poned el alias delante: `r.id_cliente`
0 filas de resultado	Texto con mayúsculas distintas o comillas dobles; condición demasiado estricta	Comprobad el texto exacto con `SELECT DISTINCT columna FROM tabla;`
Miles de filas de más tras un `JOIN`	El `ON` no une la clave con su clave	Revisad que une `id` con `id`
Se ha ejecutado todo el script	No seleccionasteis una sola consulta	Seleccionad solo la consulta y **Ctrl+Intro**
Se **ha borrado o cambiado de más** (script 05)	Faltaba el `WHERE`	**Revertir cambios** y repetid con el `SELECT` previo
`FOREIGN KEY constraint failed` o `CHECK constraint failed`	**La regla ha funcionado.** En el script 06 es lo esperado	Leed qué regla es y apuntadlo
`database is locked`	Hay otra consulta o ventana usando la base	Cerrad otras pestañas o ventanas con la base abierta y reintentad
Habéis estropeado la base	Cambios sin querer	**Revertir cambios** (sin haberlos escrito) o volved a descargar `turisdata.db`
La base se abre **vacía** (sin tablas)	Descarga incompleta o archivo equivocado	Mirad el tamaño (≈ 800 KB) y volved a descargarla con *Download raw file*; si sigue igual, importad `turisdata.sql` (paso 2 de la sección 1)

---

7. Al terminar

Lista de «ya sé…» (marcad entre los dos; si alguna casilla queda en blanco, repasadla)

- [ ] **Sé pedir columnas y filtrar filas** con `SELECT` y `WHERE`.
- [ ] **Sé unir dos o tres tablas** con `JOIN` y sé decir para qué sirve el `LEFT JOIN`.
- [ ] **Sé resumir por grupos** con `GROUP BY` y diferenciar `WHERE` de `HAVING`.
- [ ] **Sé usar una subconsulta** o una CTE para un paso intermedio.
- [ ] **Sé hacer un `INSERT`, `UPDATE` y `DELETE`** comprobando antes con un `SELECT`, y sé qué hace un `ROLLBACK`.
- [ ] **Sé decir para qué sirven** una clave primaria, una clave foránea, un `CHECK` y un índice.

Tres preguntas para hablar entre los dos (2 minutos)

1. **¿Qué ejercicio nos costó más** y qué nos ayudó a resolverlo?
2. **¿Qué error de los que aparecieron** nos puede pasar en el reto del sprint?
3. **¿Cómo le explicaríamos a alguien** la diferencia entre `WHERE` y `HAVING`, o entre `JOIN` y `LEFT JOIN`?

Antes de cerrar:

- Ejecutad la **sección de limpieza** del script 06.
- **No guardéis** cambios en las tablas originales (`reservas`, `clientes`, `alojamientos`). Si DB Browser os pregunta al cerrar, elegid **no guardar** (a menos que el docente os indique otra cosa).
- **Guardad vuestras consultas** en un archivo (los scripts ya tienen vuestro trabajo si los habéis ido rellenando).

---

8. Hoja de registro de la pareja

Copiadla y rellenadla mientras trabajáis (una por pareja).

Pareja: ____ · Fecha: __

Script	Ejercicios hechos	Quién conducía	La consulta que más nos costó (y por qué)	Lo que hemos aprendido
01				
02				
03				
04				
05				
06				

Los tres errores en los que más caímos: 1. 2. 3.

Dudas para preguntar al docente:

Para repasar la teoría

Si algo de la guía no os suena, repasad los apuntes del Sprint 3, en especial las secciones **2 a 7** (de `SELECT` a modelado e índices) y la tabla de **errores frecuentes en SQL**.